Введение в градиентный спуск.

Пицик Харитон, клуб ML

4.10.2025

Разбор полётов

Дополнительно про регуляризацию

Рассмотрим для L_1:

$\|w\|_1 = |w_1| + |w_2|$

Заранее выбирая “жёсткость” регуляризации t: $|w_1| + |w_2| \leq t$

Дополнительно про регуляризацию

На плоскости, линии уровня будут показывать линии, на которых функция потерь принимает **одинаковые значения**.

Иными словами, если точка (w_1, w_2) лежит на какой-то из линий уровня, то “передвигаясь” по этой линии, значение функции потерь не будет изменяться.
Минимум функции потерь без регуляризации достигается в центре самого маленького эллипса, образованным такой линией.

Дополнительно про регуляризацию

Таким образом, наша задача сводится к тому, чтобы найти такую точку пересечения с L_1, что:

1. Лежат внутри области регуляризации (в нашем случае - ромба);
2. Находятся на самой “маленькой” линии уровня функции потерь, которая касается область регуляризации.

Дополнительно про регуляризацию

Напомним, что в случае L_2, область регуляризатора будет являться кругом.

В силу такой формы, пересечение не может произойти в точке, где один из весов нулевой.

В отличие от L_2, L_1 норма способствует этому, т.к. имеет острые углы.

Дополнительно про регуляризацию

Давайте рассмотрим другое обоснование, почему L_1 способствует занулению признаков.

Давайте рассмотрим вектор весов $w = (1, \varepsilon)$.

Определим $0 < \delta < \varepsilon < 1$. Попробуем изменить по отдельности каждую из координат на δ, а затем посчитаем обе нормы.

Дополнительно про регуляризацию

Нам удалось получить, что L_2 норма предпочитает занулять большие веса (т.к. результат выходит больше, нежели мы меняем ε), а L_1 норме без разницы, большой вес мы меняем или маленький $\rightarrow$ существует вероятность в том числе и занулить признак, а L_2 физически не сможет этого сделать.

Введение в градиентный спуск

А оно нам надо?

Для полноценного понимания того, что стоит за волшебным словом “модель”, нам осталось понять, как у нас обновляются веса.

Оказывается, что за этим процессом стоит определённая логика, в которой мы разберёмся на примере `MSE`.
Для двух признаков, задача будет следующая:

$$\min_{w_1, w_2, b} \sum_{i=1}^{n} (y_i - w_1 x_{i1} - w_2 x_{i2} - b)^2$$

А оно нам надо?

Приведём это в матричный вид:

$$L(w) = \frac{1}{n}(y - Xw)^T(y - Xw)$$

Раскроем скобки:

$$y^T y - 2w^T X^T y + w^T X^T X w$$

$$\frac{\partial L}{\partial w} = -2X^T y + 2X^T X w$$

А оно нам надо?

Для поиска минимума приравняем производную к 0:

$$-2X^T y + 2X^T X w = 0$$

$$X^T X w = X^T y$$

$$w^* = \left(X^T X\right)^{-1} X^T y$$

Это — аналитическое решение для `MSE`.

Оно нам надо

Нам явно нужен какой-то фреймворк, который поможет решать задачи оптимизации, и более оптимально находить хорошие параметры модели.
Этот фреймворк называется **градиентный спуск**.

Для начала надо понять — а что такое градиент?

Понятие градиента

Понятие градиента позволяет нам понять, как функция меняется в заданной точке.

При условии, что дана функция f от n переменных $x_1, ..., x_n$, градиент записывается как

$$\left(\frac{\partial f}{\partial x_1}, ..., \frac{\partial f}{\partial x_n}\right)$$

Этот вектор показывает направление наискорейшего возрастания и обозначается как ∇f

Понятие градиента

Антиградиентом является $-\nabla f$ и он, соответственно, показывает направление наискорейшего убывания функции.

Если (анти)градиент равен 0, то это означает, что функция достигла экстремума. Определение градиента позволяет нам понять, в каких точках функция убывает или возрастает, а в каких точках — мы в экстремуме.

Градиентный спуск

Итак, веса будем обновлять индукционно:

1. Случайным образом инициализируем вектор весов w_0;
2. Последующие векторы вычисляем по специальной формуле:

Градиентный спуск

Итак, веса будем обновлять индукционно:

1. Случайным образом инициализируем вектор весов w_0;
2. Последующие векторы вычисляем по специальной формуле:

$$w^{(k)} = w^{(k-1)} - \eta_k \nabla Q(w)$$

η_k — `step size`, `learning rate`.

А когда останавливаться?

А когда останавливаться?

1. Когда ошибка перестаёт уменьшаться;

А когда останавливаться?

1. Когда ошибка перестаёт уменьшаться;

2. $\|w^{(k)} - w^{(k-1)}\| < \varepsilon$;

А когда останавливаться?

1. Когда ошибка перестаёт уменьшаться;

2. $\|w^{(k)} - w^{(k-1)}\| < \varepsilon$;

3. $|Q\big(w^{(k)}, X\big) - Q\big(w^{(k-1)}, X\big)| < \varepsilon$.

А когда останавливаться?

1. Когда ошибка перестаёт уменьшаться;

2. $\|w^{(k)} - w^{(k-1)}\| < \varepsilon$;

3. $|Q\big(w^{(k)}, X\big) - Q\big(w^{(k-1)}, X\big)| < \varepsilon$.

4. …

Слишком большой объём вычислений…

Слишком большой объём вычислений…

В статистике есть интересный трюк — среднее значение какой-то величины можно описывать не всей выборкой разом, а какой-то частью. Разумеется, с некоторой погрешностью.

Слишком большой объём вычислений…

В статистике есть интересный трюк — среднее значение какой-то величины можно описывать не всей выборкой разом, а какой-то частью. Разумеется, с некоторой погрешностью.
А что если мы попытаемся описать градиент не всей выбокрой, а…

Слишком большой объём вычислений…

В статистике есть интересный трюк — среднее значение какой-то величины можно описывать не всей выборкой разом, а какой-то частью. Разумеется, с некоторой погрешностью.

А что если мы попытаемся описать градиент не всей выбокрой, а… **одним** предсказанием.

Слишком большой объём вычислений…

В статистике есть интересный трюк — среднее значение какой-то величины можно описывать не всей выборкой разом, а какой-то частью. Разумеется, с некоторой погрешностью.

А что если мы попытаемся описать градиент не всей выбокрой, а… **одним** предсказанием.
Такого рода оценивание называется `Stochastic Gradient Descent (SGD)`.

— Andrej Karpathy (@karpathy) February 4, 2017

3e-4 is the best learning rate for Adam, hands down.

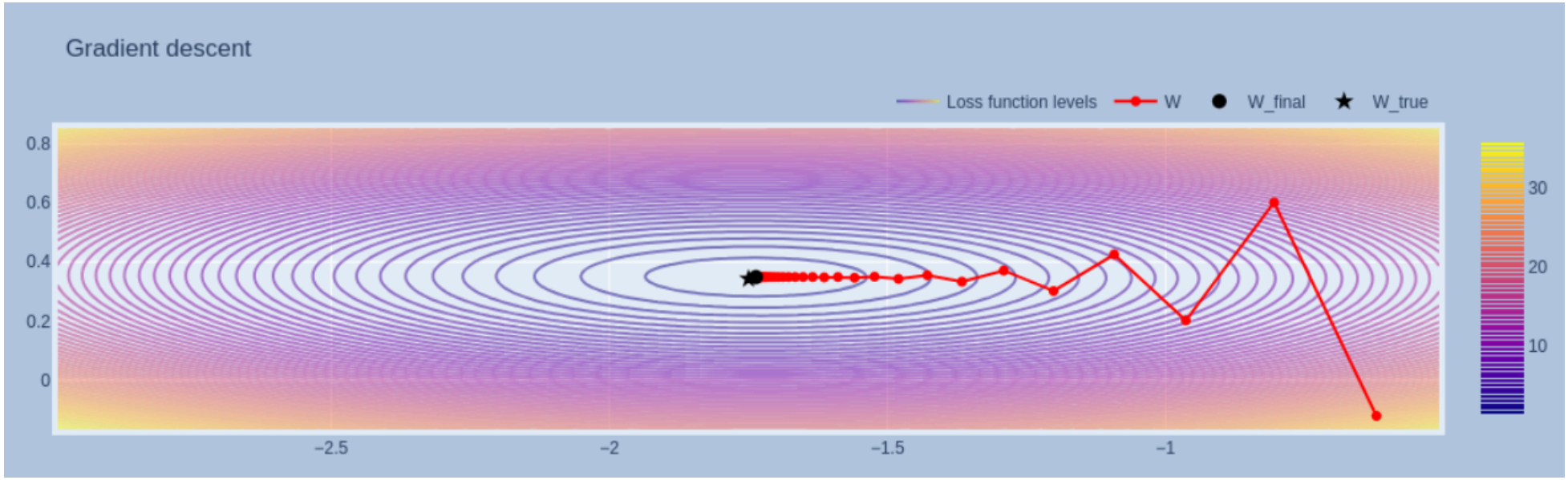
Условие Роббинса-Монро

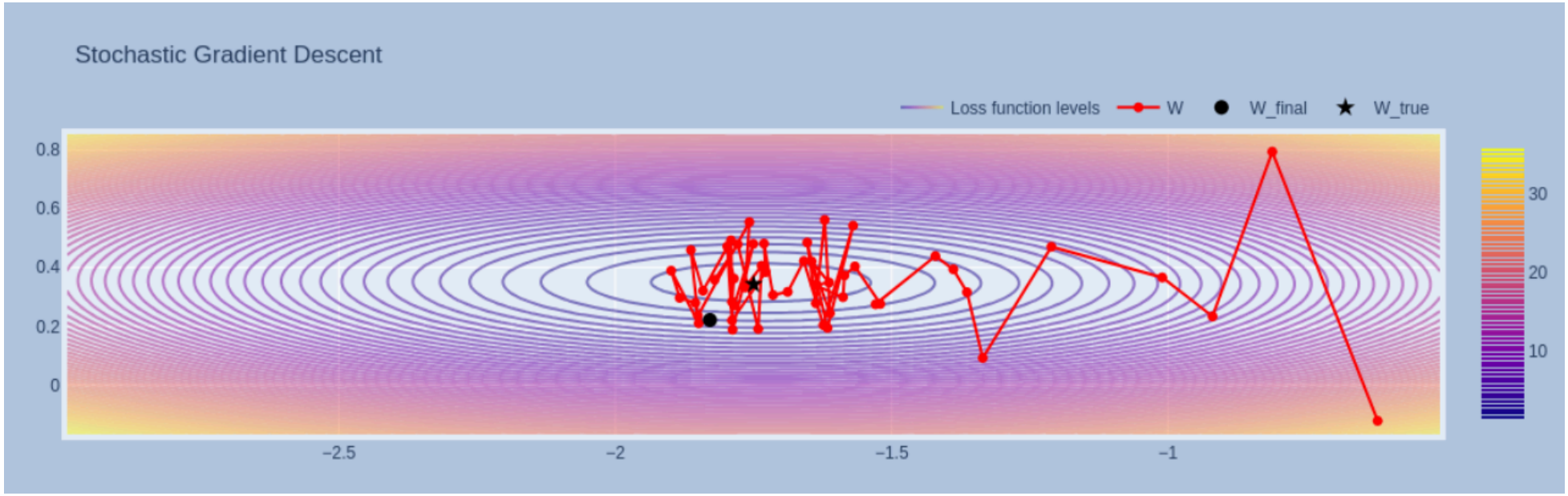
Упрощая исходную формулировку: если η_k удовлетворяет следующим условиям:

1. $\sum_{i=1}^{\infty} \eta_k = \infty$;

2. $\sum_{i=1}^{\infty} \eta_k^2 < \infty$.

то алгоритм сойдётся в минимум с вероятностью 1.


Gradient descent
Loss function levels
W
W_final
W_true
0.8
0.6
0.4
0.2
0
−2.5
−2
−1.5
−1
30
20
10


Stochastic Gradient Descent
Loss function levels
W
W_final
W_true
0.8
0.6
0.4
0.2
0
−2.5
−2
−1.5
−1
30
20
10

Модификации GD

Momentum

Идея заключается в том, чтобы хранить и использовать информацию о всех предыдущих градиентах.
Это позволит нам при шаге опираться не только на текущий градиент, но и на усреднённые предыдущие, что добавит некоторый импульс (отсюда и название метода)

Momentum

Формула выглядит следующим образом:

$$h_0 = 0$$

$$h_k = \alpha h_{k-1} + \eta_k \nabla Q\left(w^{(k-1)}\right)$$

$$w^{(k)} = w^{(k-1)} - h_k$$

Разреженные данные

Решение: `AdaGrad`

Будем для каждого объекта сохранять то, насколько часто мы уже на нём изменяли веса.

Если объект с номером g попадается впервые, то для него размер шага подбирается с чистой головой:

$$G_{kj} = G_{k-1,j} + \left(\nabla_w Q\left(w^{(k-1)}\right)\right)_j^2$$

Решение: `AdaGrad`

Будем для каждого объекта сохранять то, насколько часто мы уже на нём изменяли веса.

Если объект с номером g попадается впервые, то для него размер шага подбирается с чистой головой:

$$G_{kj} = G_{k-1,j} + \left(\nabla_w Q\left(w^{(k-1)}\right)\right)_j^2$$

$$w_j^{(k)} = w_j^{(k-1)} - \frac{\eta_k}{\sqrt{G_{kj}+\varepsilon}}\left(\nabla_w Q\left(w^{(k-1)}\right)\right)_j$$

Модификация модификации: `RMSProp`

Для решения проблемы добавим затухание:

$$G_{kj} = \alpha G_{k-1,j} + (1-\alpha)\left(\nabla_w Q\left(w^{(k-1)}\right)\right)_j^2$$

Остальное - тоже самое.